

## Fiche rapide — commande, fichier, explication

8 suites — 4 fichiers Jest — 31 tests (31/31 PASS)

Pour chaque test : la commande jest qui l'exécute seul, le chemin du fichier de test (avec les lignes), et une explication courte de ce que le test vérifie.

| tests/auth.route.test.ts |                                                                                                         |                                               |                                                                              |
|--------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------------|
|                          |                                                                                                         |                                               |                                                                              |
| 1                        | <code>npx jest tests/auth.route.test.ts -t "créé un utilisateur valide"</code>                          | <code>tests/auth.route.test.ts:32-46</code>   | Vérifie que l'inscription hache le mot de passe et crée l'utilisateur (201). |
| 2                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 400 si le body est absent"</code>                   | <code>tests/auth.route.test.ts:48-53</code>   | Rejette une requête sans corps JSON (400).                                   |
| 3                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 400 si le username est trop court"</code>           | <code>tests/auth.route.test.ts:55-62</code>   | Bloque un username de moins de 3 caractères (400).                           |
| 4                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 400 si le mot de passe est trop court"</code>       | <code>tests/auth.route.test.ts:64-71</code>   | Bloque un mot de passe de moins de 6 caractères (400).                       |
| 5                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 409 si le username existe déjà"</code>              | <code>tests/auth.route.test.ts:73-83</code>   | Traduit l'erreur Prisma P2002 en conflit 409 clair.                          |
| 6                        | <code>npx jest tests/auth.route.test.ts -t "retourne un token si les identifiants sont corrects"</code> | <code>tests/auth.route.test.ts:95-117</code>  | Vérifie le flux complet login → JWT (200 + token).                           |
| 7                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 401 si l'utilisateur n'existe pas"</code>           | <code>tests/auth.route.test.ts:119-128</code> | Message générique 401 si le username est inconnu.                            |
| 8                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 401 si le mot de passe est incorrect"</code>        | <code>tests/auth.route.test.ts:130-144</code> | Même message générique 401 si le mot de passe est faux.                      |
| 9                        | <code>npx jest tests/auth.route.test.ts -t "renvoie 400 si le body de login est invalide"</code>        | <code>tests/auth.route.test.ts:146-153</code> | Valide aussi le DTO sur la route de connexion.                               |
| 10                       | <code>npx jest tests/auth.route.test.ts -t "renvoie 500 si JWT_SECRET n'est pas configuré"</code>       | <code>tests/auth.route.test.ts:155-171</code> | Détecte une config serveur manquante avant de signer un token.               |

| tests/middlewares.test.ts |                                                                                                            |                                                |                                                           |
|---------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------|-----------------------------------------------------------|
|                           |                                                                                                            |                                                |                                                           |
| 1                         | <code>npx jest tests/middlewares.test.ts -t "laisse passer une requête avec un bearer token valide"</code> | <code>tests/middlewares.test.ts:23-33</code>   | Laisse passer si le token Bearer est valide.              |
| 2                         | <code>npx jest tests/middlewares.test.ts -t "renvoie 401 si le header authorization est absent"</code>     | <code>tests/middlewares.test.ts:35-40</code>   | Bloque une requête sans header Authorization.             |
| 3                         | <code>npx jest tests/middlewares.test.ts -t "renvoie 401 si le token est invalide"</code>                  | <code>tests/middlewares.test.ts:42-53</code>   | Intercepte un jwt.verify qui échoue (token corrompu).     |
| 4                         | <code>npx jest tests/middlewares.test.ts -t "renvoie 500 si JWT_SECRET n'est pas configuré"</code>         | <code>tests/middlewares.test.ts:55-65</code>   | Même règle de config serveur, côté middleware cette fois. |
| 5                         | <code>npx jest tests/middlewares.test.ts -t "renvoie 400 si le body est absent"</code>                     | <code>tests/middlewares.test.ts:75-80</code>   | Rejette un body manquant avant validation du DTO.         |
| 6                         | <code>npx jest tests/middlewares.test.ts -t "renvoie 400 si le body est un tableau JSON"</code>            | <code>tests/middlewares.test.ts:82-89</code>   | Rejette un body qui n'est pas un objet (tableau).         |
| 7                         | <code>npx jest tests/middlewares.test.ts -t "renvoie 400 si le body ne respecte pas le DTO"</code>         | <code>tests/middlewares.test.ts:91-98</code>   | Remonte le détail des erreurs class-validator.            |
| 8                         | <code>npx jest tests/middlewares.test.ts -t "laisse passer un body valide"</code>                          | <code>tests/middlewares.test.ts:101-109</code> | Transmet le body validé à la route suivante.              |

## tests/users.route.test.ts

| 1 | <code>npx jest tests/users.route.test.ts -t "renvoie la liste des utilisateurs"</code>                            | <code>tests/users.route.test.ts:31-41</code>   | Expose la liste des utilisateurs (200).                                 |
|---|-------------------------------------------------------------------------------------------------------------------|------------------------------------------------|-------------------------------------------------------------------------|
| 2 | <code>npx jest tests/users.route.test.ts -t "renvoie 404 si l'utilisateur n'existe pas"</code>                    | <code>tests/users.route.test.ts:45-51</code>   | 404 si l'id demandé n'existe pas.                                       |
| 3 | <code>npx jest tests/users.route.test.ts -t "renvoie 401 sans token"</code>                                       | <code>tests/users.route.test.ts:62-67</code>   | La mise à jour d'avatar est protégée par <code>requireAuth</code> .     |
| 4 | <code>npx jest tests/users.route.test.ts -t "renvoie 404 si le média demandé n'existe pas"</code>                 | <code>tests/users.route.test.ts:69-80</code>   | Empêche de lier un avatar à un média inexistant.                        |
| 5 | <code>npx jest tests/users.route.test.ts -t "met à jour l'avatar quand le média existe"</code>                    | <code>tests/users.route.test.ts:82-104</code>  | Vérifie l'appel Prisma exact et la réponse 200.                         |
| 6 | <code>npx jest tests/users.route.test.ts -t "permet de retirer l'avatar avec mediaId null"</code>                 | <code>tests/users.route.test.ts:106-127</code> | Retire l'avatar sans revalider de média ( <code>mediaId: null</code> ). |
| 7 | <code>npx jest tests/users.route.test.ts -t "renvoie 404 si l'utilisateur n'existe pas au moment du PATCH"</code> | <code>tests/users.route.test.ts:129-140</code> | Gère l'erreur Prisma P2025 (utilisateur supprimé entre-temps).          |

## tests/answers.test.ts

| 1 | <code>npx jest tests/answers.test.ts -t "renvoie correct=true et enregistre la réponse quand elle est bonne"</code> | <code>tests/answers.test.ts:21-31</code> | Bonne réponse : <code>correct=true</code> + enregistrement en base.     |
|---|---------------------------------------------------------------------------------------------------------------------|------------------------------------------|-------------------------------------------------------------------------|
| 2 | <code>npx jest tests/answers.test.ts -t "renvoie correct=false quand la réponse est mauvaise"</code>                | <code>tests/answers.test.ts:33-41</code> | Mauvaise réponse : <code>correct=false</code> , bonne réponse renvoyée. |
| 3 | <code>npx jest tests/answers.test.ts -t "renvoie null si la question n'existe pas"</code>                           | <code>tests/answers.test.ts:43-50</code> | Aucun enregistrement si la question est inconnue.                       |
| 4 | <code>npx jest tests/answers.test.ts -t "vérifie que l'utilisateur existe quand un userId est fourni"</code>        | <code>tests/answers.test.ts:52-64</code> | Contrôle l'existence de l'utilisateur avant d'enregistrer.              |
| 5 | <code>npx jest tests/answers.test.ts -t "renvoie null si l'utilisateur fourni n'existe pas"</code>                  | <code>tests/answers.test.ts:66-74</code> | N'enregistre rien pour un utilisateur fantôme.                          |
| 6 | <code>npx jest tests/answers.test.ts -t "n'essaie pas de charger un utilisateur quand userId est absent"</code>     | <code>tests/answers.test.ts:76-83</code> | Évite un appel base inutile en mode anonyme.                            |